

Competitive Modeling Playbook for BKBC ATI Prediction From SomaScan Proteomics

What the BKBC ATI proteomics paper implies about signal and pitfalls

The 2024 Nature Communications ¹ BKBC study (“Plasma proteomics of acute tubular injury”) is unusually actionable for a hackathon because it essentially tells you three things at once: (1) the raw feature space is very high-dimensional (6,592 plasma proteins), (2) there is strong multivariable, multiple-testing-robust signal tied to ATI severity (156 proteins after adjustment and correction), and (3) that signal persists even after adjusting for key clinical covariates such as age/sex/race and eGFR, meaning the proteomic panel is not merely a trivial proxy for kidney function labs. ²

From a competition perspective, this is the “sweet spot” problem shape: medium-N (~1,500 in your hackathon) with strong biological effect sizes on a subset of features, but with many correlated/irrelevant features that punish sloppy modeling and reward disciplined feature selection + ensembling. The BKBC paper explicitly reports that some proteins are measured by multiple aptamers (170 aptamers for 156 proteins), which is a common source of redundancy/collinearity and a direct reason simple “feed everything to a model” strategies can become unstable and seed-sensitive. ²

The clinical covariates are both a gift and a trap. They are a gift because eGFR and other pre-biopsy covariates are likely highly predictive and will boost AUROC quickly; they are a trap because (a) they can dominate model behavior, masking whether your proteomics modeling is actually robust, and (b) if the held-out test split differs in case mix or eGFR distribution, over-reliance can reduce generalization. The BKBC paper’s approach of identifying proteins *after adjusting for eGFR* is a strong hint for you: build models that can still perform when eGFR is down-weighted or even removed, and then add covariates back in as an ensemble member rather than letting them single-handedly drive predictions. ²

State of the art for medium-N, high-p proteomics classification

Tree ensembles remain the competition workhorse in this regime

Across large, careful tabular benchmarks, gradient-boosted decision trees (GBDTs; XGBoost/LightGBM/CatBoost family) remain extremely hard to beat on typical medium-sized tabular datasets, even when deep models are tuned heavily. Two widely cited benchmarking efforts conclude that tree-based methods remain state-of-the-art in common tabular regimes and that neural nets often require more tuning and are more sensitive to uninformative features. ³

This matters even more in omics settings because omics matrices often include thousands of features with (i) highly correlated blocks, (ii) many weakly informative or irrelevant features, and (iii) modest sample sizes relative to p. The “tree models outperform tabular DL” observation is consistent with the fact that the BKBC

proteomic signal is concentrated in a subset (156 proteins), while many other features likely act as noise or near-noise. ⁴

Proteomics papers that actually benchmark models frequently pick boosting for generalization

A recent large-scale proteomics modeling example (proteomic age clock) compared penalized linear models, gradient boosting (LightGBM), and multiple neural architectures (MLP/ResNet/TabR) trained on thousands of proteins; LightGBM was selected as the final model because it generalized best across independent cohorts, and the authors explicitly combine boosting with Boruta-style selection and SHAP-driven refinement. ⁵

This is directly analogous to your setting: “many proteins, need generalization.” Even though their task is regression (age), the modeling lesson transfers because the core risks (high-p, correlated features, overfitting, cross-cohort robustness) are the same. ⁵

Deep tabular models can help, but rarely as the single best hackathon bet

Modern deep tabular models (e.g., FT-Transformer, SAINT, TabNet, TabR) are legitimate and sometimes strong, but the literature and benchmarks suggest they are less consistently dominant than GBDTs in typical tabular settings—especially without careful tuning and regularization. ⁶

TabR in particular is notable because it’s a retrieval-augmented deep model that reports strong average performance and can outperform GBDTs on some benchmarks, making it one of the more practical “deep tabular” options to try if you already have a stable GBDT baseline. ⁷

A key practical constraint for you is time-to-iteration: deep tabular models often require more careful preprocessing (scaling, regularization, early stopping discipline) and more hyperparameter sensitivity than GBDTs, which can slow down progress in a 24-hour event. ⁸

Tabular foundation models (TabPFN, TabICL) are real—and very relevant—but you must adapt them to high-p proteomics

Tabular foundation models (TFMs) using in-context learning (ICL), especially TabPFN, represent a genuinely new competitive lever for small-to-medium tabular tasks. The 2025 Nature ⁹ TabPFN paper reports that TabPFN is designed for small-to-moderate datasets and shows dominant performance on benchmarks constrained to $\leq 10,000$ samples and ≤ 500 features, with strong out-of-the-box performance (often beating tuned GBDTs) and explicit guidance that for larger datasets or certain regimes, CatBoost/XGBoost/AutoGluon may outperform it. ¹⁰

TabICL (ICML 2025 / arXiv 2025) is positioned as a scalable tabular foundation model for classification that aims to extend ICL to larger tables, and it still frames its operational regime around feature counts on the order of hundreds (explicitly discussing “up to 500 features” while scaling rows much larger). ¹¹

For your problem, the immediate implication is:

- TFMs are attractive because your N (~1,500) is in their “small-to-medium” comfort zone.
- Your p (~6,592 proteins + covariates) is far outside the feature count regime emphasized in the TabPFN/TabICL papers.
- Therefore, TFMs become most useful **after** aggressive feature reduction (e.g., 200–500 proteins) as a fast, strong additional model in an ensemble, not as your single main model. ¹²

Feature selection strategies that are fast, stable, and competition-effective

High-dimensional proteomics is a feature selection problem as much as it's a classifier problem. A 2025 high-dimensional proteomics ML workflow paper emphasizes that *robust feature selection is critical for model stability and performance*, and it explicitly frames feature selection as a central determinant of results in proteomics modeling pipelines. ¹³

Biomarker-first priors are uniquely justified here

Unlike most competitions, you have a direct, cohort-matched literature prior: the BKBC paper's 156 proteins associated with ATI—plus a ranked set of strong positives (SPP1/osteopontin, MRC1, TNC, NTN4, MZB1) and additional biologically relevant markers including KIM-1/HAVCR1. Starting with these proteins is not only fast; it is evidence-based. ²

The most practical way to use this prior in a hackathon is not to hard-lock to 156 proteins, but to treat it as the first “feature subset ladder rung”:

- Subset A: clinical covariates only
- Subset B: clinical + 156 ATI proteins
- Subset C: clinical + 156 + top- k from your own training set ranking (within CV)
- Subset D: clinical + data-driven selected set (Boruta/SHAP/elastic net stability)

This provides fast baselines and produces rapid signal about whether the leaderboard is dominated by clinical covariates, proteomic signal, or their interaction. ¹⁴

Elastic net / LASSO screening is fast and often useful, but unstable without safeguards

Penalized linear models (LASSO/elastic net logistic regression) are popular in omics because they can handle $p \gg n$ and yield sparse models. However, modern omics methodology work emphasizes that L1-based sparsity can be *unstable* under small perturbations in data—selected feature sets can vary widely—especially when features are correlated, which is typical in proteomics. ¹⁵

A strong 2024 Nature Biotechnology ¹⁶ paper introduces Stabl specifically to address this reliability issue in high-dimensional omics by combining noise injection with data-driven thresholding to produce sparse *and reliable* biomarker sets while maintaining predictive performance, and demonstrates it can distill very high-dimensional datasets (1,400–35,000 features) down to short biomarker lists. ¹⁵

Hackathon translation: if you use elastic net as a screening step, add a “stability” wrapper (repeat subsampling, count selection frequency) so you don’t overfit to one lucky split. ¹⁷

Boruta + SHAP selection is a proven proteomics pattern, and it’s fast enough to emulate

The proteomic age clock paper describes using Boruta-style selection with SHAP values to identify relevant proteins and then recursive feature elimination using SHAP to shrink to smaller panels (including a 20-protein panel achieving most of the performance). ⁵

This is a very competition-adaptable recipe because it is: - Model-agnostic in spirit (GBDT-based) - Able to handle nonlinearity and interactions - Very implementable in Python quickly - Naturally produces ranked feature lists for later modeling and interpretation ⁵

SHAP-based feature selection is also increasingly formalized in recent methodological literature as a general feature-ranking approach (select top-k by SHAP importance) that can be used across modeling settings. ¹⁸

Avoiding feature-selection leakage is non-negotiable

If you do feature selection on the full dataset before cross-validation, you can produce severely optimistic performance estimates. This is well documented: a methodological study shows incorrect feature selection outside CV creates biased results, and scikit-learn’s official “common pitfalls” documentation explicitly warns that feature selection (and most preprocessing) must be fit on training folds only to avoid leakage. ¹⁹

That warning is especially relevant in your dataset because p is large and the probability of picking spurious correlates by chance is non-trivial; leakage will mislead you into believing you have a leaderboard-winning jump when you do not. ¹⁹

Ensembling and stacking strategies that reliably produce leaderboard gains

The evidence base for “GBDT + something else” ensembles is strong

A widely cited empirical paper (“Tabular data: deep learning is not all you need”) reports that while XGBoost often outperforms deep tabular models on many datasets, an *ensemble of deep models and XGBoost* can improve over XGBoost alone. This aligns closely with Kaggle-style best practice: use GBDTs as the anchor, add diversity carefully, and ensemble. ²⁰

TabPFN’s own Nature paper also places it in an ensemble-aware framing: it claims strong out-of-the-box performance and discusses AutoGluon-style stacked ensembles as a competing approach, emphasizing that top performance on real-world problems still benefits from data-scientist “toolkit” thinking rather than one model alone. ¹⁰

Competition-grade stacking basics you should copy

Stacking (using out-of-fold predictions from multiple base models as inputs to a meta-model) is a standard way to improve tabular competition scores without overfitting, provided you generate meta-features correctly with OOF predictions. ²¹

Kaggle's own community education pages define out-of-fold predictions as a core concept used to validate performance and commonly used with ensemble/stacking methods. ²²

Practical implication for your hackathon: you should treat stacking as the “final 10–15%” effort—only after you have stable base models—because it's easy to accidentally leak if you shortcut OOF generation. ²³

Which ensemble components are most valuable for your exact data shape

Given $p \approx 6,592$ and $N \approx 1,500$, the most reliable diversity usually comes from combining: - A GBDT model trained on a moderately sized feature set (hundreds to a few thousand) with strong regularization and feature subsampling - A sparse linear model (elastic net logistic regression) trained on standardized features, often after screening - A “fast strong” tabular foundation model (TabPFN) trained on a heavily reduced feature set (≤ 500 features) as an additional viewpoint, if your environment supports it ²⁴

Deep tabular models like TabR can add diversity too, but they are usually not the most time-efficient way to get a dependable gain unless someone on your team has already implemented them. ²⁵

Validation design and leakage resistance when covariates can be near-proxies

Split discipline: patient-level grouping and “hidden duplicates” checks

If the dataset contains repeated measurements per patient (multiple samples/visits) or any form of duplicate rows (common in cohorts), you must do group-wise splits by patient ID. If you don't have an explicit patient ID, you should still check for exact duplicates and near-duplicates (hash rows, check identical profiles) before you trust CV. This is not “nice to have”; it is how you prevent information bleed between folds in clinical cohorts. (The general principle—preprocessing and model selection must be isolated within training folds—applies here.) ²⁶

Fold construction for ATI: preserve base rates and stress-test generalization

For binary ATI, use stratified k-fold CV. For ordinal severity labels (e.g., none/mild/moderate/severe), still use a stratification strategy that preserves the distribution across folds, because the BKBC paper reports substantial class imbalance (e.g., severe ATI is rare). ²

Measuring “covariate proxy risk” explicitly

Your core risk is that models win CV by learning a clinical proxy rather than a robust molecular signature. The BKBC paper shows proteins remain associated after adjusting for eGFR, strongly suggesting there is meaningful signal beyond kidney function. ²

To ensure your model isn't brittle, you should routinely compute three CV AUCs: - Clinical-only model - Proteomics-only model (at least on the 156 ATI proteins) - Combined model

If combined performance is only marginally better than clinical-only, you should expect domain shift to hurt you more, and you should focus on identifying proteomics features that add incremental value. This "ablation" practice follows directly from the fact that the published analysis treats eGFR as an adjustment factor rather than the target itself. ²

Feature selection and preprocessing must occur inside each CV fold

This bears repeating because it is the most common way teams fool themselves. If you standardize, impute, select features, or reduce dimensions, those steps must be fit on the training fold and applied to the validation fold. Both methodological studies and scikit-learn guidance document that failing to do this biases results. ¹⁹

Concrete under-24-hour implementation plan with hyperparameter starting points

This is a prioritized plan optimized for "fast path to high AUROC + reliable generalization." It assumes: you have Python, standard ML packages, and a single strong GPU (GPU is optional for most steps). It is designed so that you can stop early and still have a competitive submission.

Phase setup: first 60–90 minutes

Start by operationalizing the BKBC prior and building a leak-resistant evaluation harness.

Data quick checks and setup: - Confirm target format (binary vs ordinal). If ordinal, decide your operational target as either: - binary: (moderate/severe) vs (none/mild) or similar, or - continuous severity score prediction with AUROC computed on a derived binary label (many competitions accept any monotonic score). The BKBC paper uses semiquantitative severity categories, so this is plausible. ² - Identify patient ID if present; if absent, check for duplicates (exact row duplicates; high cosine similarity clusters). Use group CV if IDs exist. - Build a robust CV function that: - uses StratifiedKFold (binary) or StratifiedKFold on severity bins (ordinal), - fits all preprocessing and feature selection inside each fold, and - outputs OOF predictions for stacking later. ²⁷

Feature subsets you should implement immediately: - **FS0 (Clinical-only)**: all clinical covariates. - **FS1 (Published ATI set)**: clinical + the 156 proteins (and all aptamers corresponding to them, if your dataset is aptamer-level). - **FS2 (Expanded)**: clinical + FS1 + top-k proteins selected inside each fold by a fast filter (e.g., univariate AUC or mutual information), with k initially 500–1,500. Use this only as an intermediate step (do not trust it without fold-wise selection). ²⁸

Phase one: establish hard-to-beat baselines in 2–4 hours

Your objective is to get 3–4 models that are individually strong and stable.

Model A: LightGBM (anchor model) - Why: strong tabular generalization in proteomics-scale settings, used as the final model in a large proteomic benchmark due to cross-cohort robustness. ²⁹ - Starting hyperparameters (good default for high-p, medium-N): - objective: binary - metric: auc - boosting: gbd - n_estimators: 5000 (use early stopping) - learning_rate: 0.02–0.05 (start 0.03) - num_leaves: 31 or 63 (start 63) - min_data_in_leaf: 20–80 (start 50) - feature_fraction: 0.15–0.35 (start 0.20) to handle many features - bagging_fraction: 0.7–0.9 (start 0.8), bagging_freq: 1 - lambda_l1: 0.0–1.0 (start 0.1), lambda_l2: 1.0–10.0 (start 5) - max_depth: -1 (or cap at 6–10 if overfitting) - early_stopping_rounds: 100–200 - Train on FS1 first, then FS2 if stable.

This design mirrors common proteomics boosting setups and aligns with the proteomic aging clock paper's use of high-capacity boosting plus early stopping and feature selection refinement. ⁵

Model B: XGBoost (secondary GBDT for diversity) - Why: complementary inductive biases sometimes help; ensemble of tree models is standard in tabular competitions. ³⁰ - Starting hyperparameters: - tree_method: hist - eval_metric: auc - n_estimators: 4000–8000 with early stopping (200) - eta: 0.02–0.05 (start 0.03) - max_depth: 3–6 (start 4) - min_child_weight: 5–20 (start 10) - subsample: 0.7–0.9 (start 0.8) - colsample_bytree: 0.15–0.35 (start 0.25) - reg_alpha: 0.0–1.0 (start 0.1) - reg_lambda: 1.0–10.0 (start 5) - gamma: 0–1 (start 0) - Train on FS1 and FS2.

Model C: Elastic net logistic regression (sparse linear baseline + screening tool) - Why: strong in high-dimensional omics for fast baselines and as a feature-screening mechanism; but must be stabilized due to selection instability in correlated omics. ³¹ - Procedure: - Standardize features inside fold. - Use saga solver (scikit-learn) with l1_ratio in {0.1, 0.5, 0.9} and C grid (logspace). - Use class_weight='balanced' if imbalance is high. - Outputs: - OOF predictions for stacking. - Selected feature set (non-zero coefficients) per fold for stability analysis.

Model D (optional but high-upside): TabPFN on reduced features - Why: extremely strong out-of-the-box in its intended regime ($\leq 10k$ samples, ≤ 500 features), which your N satisfies; use it after reducing p. ¹⁰ - How to make it fit your proteomics: - Build a ≤ 300 –500 feature matrix per fold: - include all clinical covariates (if few), - include the 156 ATI proteins, and - if still under the limit, add the top-k by fold-wise SHAP importance from LightGBM. - Run TabPFN to generate predictions; no heavy tuning expected. - If feature count is still too high, shrink further with SHAP-ranked selection.

TabICL note: it's an ICL foundation model designed to scale to large row counts, but it still centers typical feature counts around the hundreds; in a hackathon, TabICL is most realistically a “read the paper for concepts” rather than something you implement unless the organizers provide it directly. ¹¹

Phase two: feature selection that actually generalizes in 3–6 hours

You should now focus on building one high-quality feature set for your GBDT anchor, plus a smaller feature set for TabPFN.

Step one: Biomarker-first + SHAP refinement (fast, evidence-backed) - Train LightGBM on FS1. - Compute SHAP values (or LightGBM gain importance if SHAP is too slow). - Build FS1-SHAP(k): keep top k proteins by $\text{mean}(|\text{SHAP}|)$ across folds (k in {200, 400, 800}). This matches a published pattern: the proteomic age clock uses SHAP and Boruta to identify relevant proteins and then uses SHAP-driven recursive elimination to form smaller panels. ⁵

Step two: Boruta-style relevance filtering (strong if you have time) - Implement Boruta-like selection (shadow/permutated features) using LightGBM importance: - For each fold, append shadow features (permuted versions), fit a model, and keep features consistently above shadow maxima. This is directly aligned with the proteomic age clock's described Boruta procedure using shadow features and SHAP thresholds. ⁵

Step three: Stability wrapper (recommended) - Repeat fold-wise selection across 3–5 different CV seeds and keep proteins that appear in ≥ 60 –80% of runs. This is a lightweight version of the reliability-driven motivation in Stabl: reducing sensitivity to perturbations in $p \gg n$ omics selection. ¹⁵

Critical constraint: every one of these selection steps must be executed within CV folds to avoid leakage. ²⁶

Phase three: blending and stacking in 2–5 hours

Do not overcomplicate. Your goal is predictable gains.

Step one: seed averaging (almost free gain) - For LightGBM and XGBoost, run 3–5 random seeds (random_state changes, plus feature_fraction/colsample randomness) and average predictions. This reduces variance and usually yields small but consistent AUC improvements in tabular competitions. ³²

Step two: simple weighted blending (baseline ensemble) - Start with a convex blend of probabilities: - 0.55 LightGBM + 0.25 XGBoost + 0.20 elastic net - Tune weights by CV on OOF predictions (grid over 0.05 steps). This aligns with the general evidence that combining diverse models can outperform single anchors on tabular tasks. ²⁰

Step three: stacking (only if you have stable OOF and time) - Construct OOF predictions for each base model using the same fold splits. - Fit a ridge-regularized logistic regression meta-model on the OOF predictions (and optionally on a small subset of clinical covariates). - Predict test set by averaging meta-model outputs trained per fold. Use this only if your OOF pipeline is correct; stacking is powerful but easy to leak if done incorrectly, which is why Kaggle emphasizes OOF predictions for stacking. ²¹

If you include TabPFN, treat it as just another base model: - Train TabPFN within each fold using the reduced feature set and generate OOF predictions. - Add to blend/stack; do not replace your GBDT anchor unless it clearly wins CV. ³³

Phase four: “model audit” before final submission in the last 1–2 hours

This is how you avoid catastrophic generalization mistakes.

Ablations you should run: - Clinical-only vs proteomics-only vs combined (ensure combined is not just clinical). - Remove eGFR (and creatinine if present): does performance collapse? If yes, you're over-reliant on a proxy; put more effort into proteomic feature sets that add robust incremental value. The published BKBC result that proteins remain significant after eGFR adjustment suggests it should not collapse completely if you've captured real signal. ²

Sanity checks: - Confirm no preprocessing is fit on full data before CV (scaling, selection, PCA). - Confirm you did not accidentally leak labels in target encoding or imputation steps. These are standard leakage modes documented in both methodological studies and scikit-learn guidance. ¹⁹

Interpretability quick pass (optional but useful for judges): - Report top 10 proteins by SHAP from your final LightGBM model and note overlap with the BKBC paper's known top proteins (SPP1, MRC1, TNC, and inclusion of KIM-1/HAVCR1). This not only improves trust; it is also a quick check that your model learned plausible biology rather than artifacts. ²

¹ ¹⁸ A problem-agnostic approach to feature selection and ...

https://link.springer.com/article/10.1186/s40537-024-01041-1?utm_source=chatgpt.com

² ⁴ ¹⁴ ²⁸ Plasma proteomics of acute tubular injury | Nature Communications

<https://www.nature.com/articles/s41467-024-51304-x>

³ Why do tree-based models still outperform deep learning ...

https://papers.neurips.cc/paper_files/paper/2022/file/0378c7692da36807bdec87ab043cdadc-Paper-Datasets_and_Benchmarks.pdf?utm_source=chatgpt.com

⁵ ⁹ ²⁹ Proteomic aging clock predicts mortality and risk of common age-related diseases in diverse populations | Nature Medicine

<https://www.nature.com/articles/s41591-024-03164-7>

⁶ Revisiting Deep Learning Models for Tabular Data

https://arxiv.org/pdf/2106.11959?utm_source=chatgpt.com

⁷ ²⁵ TABR: TABULAR DEEP LEARNING MEETS NEAREST ...

https://proceedings.iclr.cc/paper_files/paper/2024/file/4ef594af0d9a519db8fb292452c461fa-Paper-Conference.pdf?utm_source=chatgpt.com

⁸ ²⁰ ³⁰ Tabular data: Deep learning is not all you need

https://www.sciencedirect.com/science/article/abs/pii/S1566253521002360?utm_source=chatgpt.com

¹⁰ ¹² ²⁴ ³³ Accurate predictions on small data with a tabular foundation model | Nature

<https://www.nature.com/articles/s41586-024-08328-6>

¹¹ TabICL: A Tabular Foundation Model for In-Context Learning on Large Data

<https://arxiv.org/html/2502.05564>

¹³ Applying machine learning to high-dimensional proteomics ...

https://pmc.ncbi.nlm.nih.gov/articles/PMC11874791/?utm_source=chatgpt.com

¹⁵ ¹⁷ ³¹ Discovery of sparse, reliable omic biomarkers with Stabl | Nature Biotechnology

<https://www.nature.com/articles/s41587-023-02033-x>

¹⁶ ²⁶ ²⁷ 11. Common pitfalls and recommended practices

https://scikit-learn.org/stable/common_pitfalls.html?utm_source=chatgpt.com

¹⁹ Measuring the bias of incorrect application of feature selection ...

https://pmc.ncbi.nlm.nih.gov/articles/PMC8613324/?utm_source=chatgpt.com

²¹ ²² ²³ Out of Fold predictions(OOF)

https://www.kaggle.com/getting-started/177975?utm_source=chatgpt.com

32 **Grandmaster Pro Tip: Winning First Place in a Kaggle ...**

https://developer.nvidia.com/blog/grandmaster-pro-tip-winning-first-place-in-a-kaggle-competition-with-stacking-using-cuml/?utm_source=chatgpt.com